TP : Transfer Learning et Apprentissage Multitâche

Compte-rendu des exercices 1, 2 et 3 · analyse comparative et recommandations

M2 SDIA — Advanced Artificial Intelligence · ENSET

[Noms des étudiants] · Octobre 2026

Résultats principaux

- **Exercice 1 (fleurs).** CNN de zéro : **72,3 %** → MobileNetV2 en extraction de caractéristiques : **90,2 %** (seulement 6 405 paramètres entraînés) → fine-tuning de 30 couches : **90,6 %** (+0,4 pt, non significatif, avec sur-apprentissage). Avec 5 % des données (146 images), le transfert (75,3 %) dépasse déjà le CNN de zéro entraîné sur 100 % (73,7 %).
- **Exercice 2 (backbones).** EfficientNetB0 est le plus précis (**93,9 %**) ; MobileNetV2 est le plus compact (**9,2 Mo**) et le plus rapide (19,5 ms/image à batch = 1, CPU). VGG16 et ResNet50 sont dominés.
- **Exercice 3 (multitâche).** Le tronc partagé réduit les paramètres de **50 %** et la latence de **38 à 50 %**, à précision équivalente (écarts ≤ 0,3 pt, dans le bruit inter-graines). Un λ_B élevé (5) provoque un léger transfert négatif sur la tâche A (−0,8 pt).
- **Décalage de distribution.** Le modèle fine-tuné est fragile : flou −41,7 pts, bruit −20,2 pts, « nuit » −47,5 pts d'accuracy.

Environnement et protocole expérimental

Python 3, TensorFlow 2.21 (exécution sur **CPU**, sans GPU), scikit-learn, pandas, matplotlib. Graine 42 pour les exercices 1 et 2 ; **3 graines** (42, 59, 76) pour l'exercice 3, avec résultats en moyenne ± écart-type. Le **code complet et commenté** se trouve dans le notebook *TP_Transfer_Learning_Multitache_Complet.ipynb* ; ce rapport en reprend les extraits essentiels, les courbes, les tableaux et les analyses.

Précision statistique. La validation compte 734 images : à ≈ 90 % d'accuracy, l'erreur-type est ≈ 1,1 pt (intervalle à 95 % ≈ ± 2,2 pts). Un écart inférieur à ~2 pts entre deux modèles, mesuré sur une seule exécution, n'est donc pas concluant ; nous le signalons à chaque fois. Les écarts de 15 à 35 pts entre CNN de zéro et transfert sont, eux, très nets.

Correspondance avec l'énoncé

Énoncé	Contenu	Section
Exercice 1	Données, CNN de zéro, extraction, fine-tuning, quantité de données, prétraitement	1
Exercice 2	Quatre backbones : tableau comparatif, t-SNE, Top-1/Top-5 ImageNet, recommandations	2
Exercice 3	Multitâche (schéma), mono vs multi, λ_B, peu de données, cohérence	3
Analyse transversale	Tableau récapitulatif, six questions, robustesse, recommandations, limites	4

1. Exercice 1 — Identification de fleurs par transfer learning

1.1 Données et répartition des classes

Jeu *flower_photos* : 3 670 photos, 5 classes (marguerite, pissenlit, rose, tournesol, tulipe), redimensionnées en 160×160. Découpage **stratifié 80 / 20** : 2 936 images d'entraînement, 734 de validation. Effectifs : pissenlit 898, tulipe 799, tournesol 699, rose 641, marguerite 633 (rapport max/min ≈ 1,4 : déséquilibre modéré). Référence à dépasser : 24,5 % pour un classifieur constant « pissenlit », 20 % pour le hasard. Les photos sont très hétérogènes (cadrage, lumière, fond, bouquets, vases), ce qui annonce les confusions rose ↔ tulipe observées plus loin.

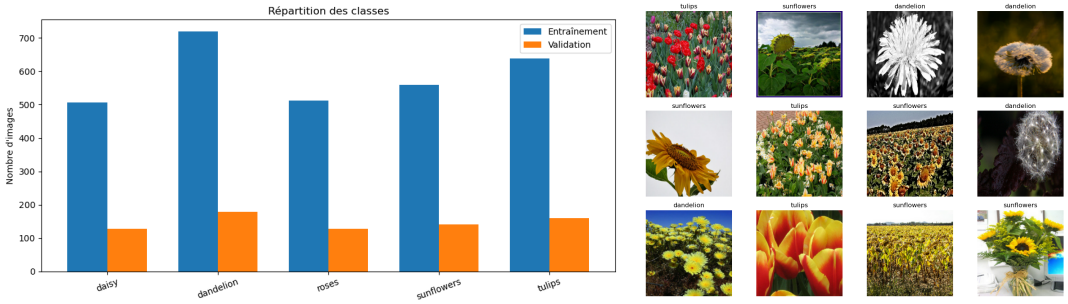


Figure 1 — Répartition des classes (gauche) et exemples d'images d'entraînement avec leur étiquette (droite).

```
# Découpage stratifié 80/20 et pipeline tf.data (decode_image + resize 160x160)
train_paths, val_paths, y_train, y_val = train_test_split(paths, labels, test_size=0.20,
                                                          random_state=SEED, stratify=labels)
train_ds = make_dataset(train_paths, y_train, training=True)
val_ds   = make_dataset(val_paths,   y_val,   training=False)
```

1.2 Référence : CNN entraîné à partir de zéro

Architecture *blocs4* du TP3 adaptée (entrée RGB 160×160×3, sortie softmax à 5 classes) : **913 221 paramètres**, tous entraînés. Résultat après 10 époques : **72,3 %** de validation (loss 0,825). Il s'agit d'un **sous-apprentissage** plutôt que d'un sur-apprentissage : l'accuracy d'entraînement (74,7 %, mesurée avec augmentation et dropout actifs) est quasi égale à celle de validation et les deux courbes montent encore à l'époque 10. Ce modèle constitue le plancher de référence.

```
# CNN de référence « blocs4 » : augmentation intégrée au modèle (active à l'entraînement seulement)
x = layers.RandomFlip("horizontal")(inputs); x = layers.RandomRotation(0.08)(x); x = layers.RandomZoom(0.10)(x)
x = layers.Rescaling(1./255)(x)
for f in (32, 64, 64, 64):
    x = layers.Conv2D(f, 3, padding="same", activation="relu")(x); x = layers.MaxPooling2D(2)(x)
x = layers.Flatten()(x); x = layers.Dense(128, activation="relu")(x); x = layers.Dropout(0.3)(x)
outputs = layers.Dense(5, activation="softmax")(x)          # Adam 1e-3, 10 époques, batch 32
```

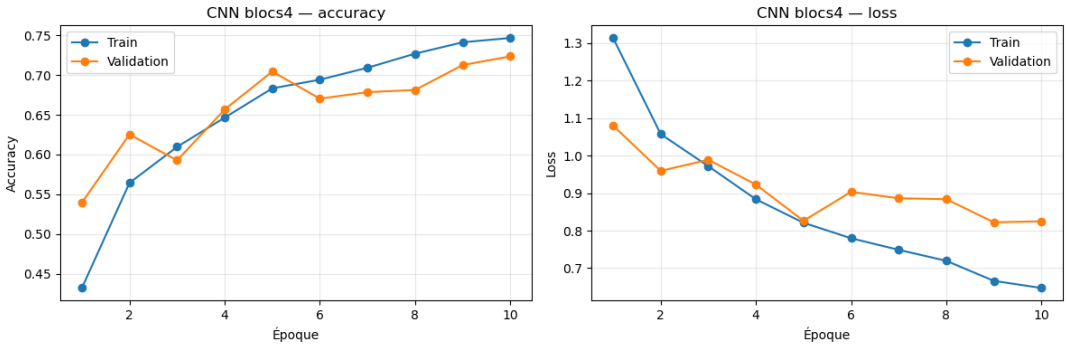


Figure 2 — CNN blocs4 depuis zéro : accuracy et loss (entraînement / validation).

1.3 Extraction de caractéristiques (MobileNetV2 gelé)

Paramètres	Total	Entraînables	Non entraînables
MobileNetV2 + GAP + Dropout(0,2) + Dense(5)	2 264 389	6 405 (0,28 %)	2 257 984

Seule la couche Dense(5) est entraînée (1 280 × 5 + 5 = 6 405 paramètres, soit 140 fois moins que le CNN de zéro). Résultat : **90,2 %** (loss 0,312), soit **+17,9 pts**. Dès la 1re époque la validation atteint 84,2 % (53,9 % pour le CNN de zéro). La validation plafonne vers 88–90 % dès l'époque 3 alors que l'entraînement monte à 95 % : le facteur limitant est la qualité des caractéristiques, pas la capacité de la tête.

```
# Extraction de caractéristiques : MobileNetV2 gelé + nouvelle tête (preprocess_input : [0,255] -> [-1,1])
base_model = MobileNetV2(weights="imagenet", include_top=False, input_shape=(160, 160, 3))
base_model.trainable = False
x = base_model(inputs, training=False)                      # BatchNorm en mode inférence
x = layers.GlobalAveragePooling2D()(x); x = layers.Dropout(0.2)(x)
outputs = layers.Dense(5, activation="softmax")(x)          # Adam 1e-3, 10 époques
```

1.4 Fine-tuning : dégel des 30 dernières couches (lr = 10^{-5})

Après dégel, **1 517 125 paramètres** sont entraînables (≈ 67 % du modèle), car les dernières couches concentrent beaucoup de canaux ; les BatchNormalization restent gelées. Résultat : **90,6 %** (loss 0,315) contre 90,2 % avant, soit **+0,4 pt ≈ 3 images sur 734** : très inférieur à l'incertitude de ± 2,2 pts, donc **non significatif**. L'accuracy d'entraînement passe de 95 % à **99,6 %** tandis que la validation reste bloquée à 90–91 % et que la loss de validation ne baisse plus (minimum 0,299, puis ≈ 0,315) : le dégel sert surtout à mémoriser l'entraînement. Aucune discontinuité à la frontière : le learning rate de 10^{-5} assure une transition douce.

```
# Fine-tuning : 30 dernières couches du tronc (BatchNorm laissées gelées), lr = 1e-5, 10 époques
for layer in base_model.layers: layer.trainable = False
for layer in base_model.layers[-30:]:
    if not isinstance(layer, layers.BatchNormalization): layer.trainable = True
mnet.compile(optimizer=keras.optimizers.Adam(1e-5), loss="sparse_categorical_crossentropy", metrics=["accuracy"])
history_fine = mnet.fit(train_mnet_ds, validation_data=val_mnet_ds, epochs=10)
```

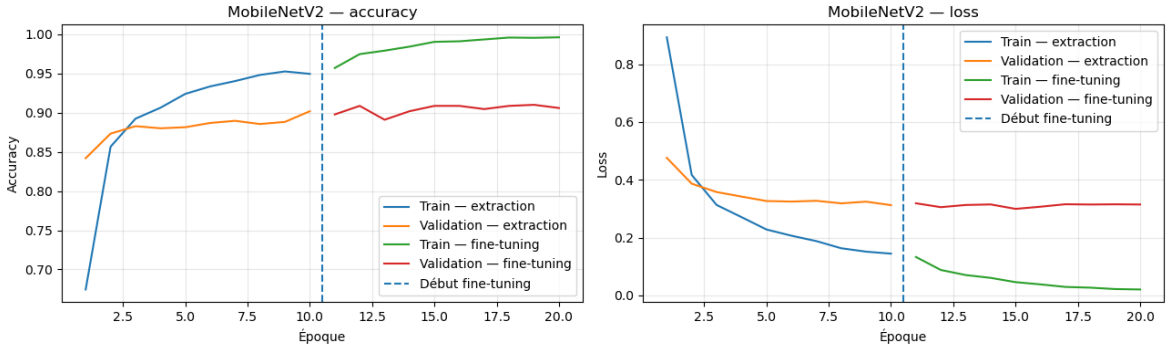


Figure 3 — Courbes continues sur 20 époques : extraction (époques 1–10) puis fine-tuning (11–20, ligne pointillée = début du dégel).

1.4b Combien de couches dégeler ? Pourquoi un learning rate faible ?

Expérience complémentaire : on repart des mêmes poids (fin de la phase « tête ») et on varie le nombre de couches dégelées et le learning rate (5 époques par configuration).

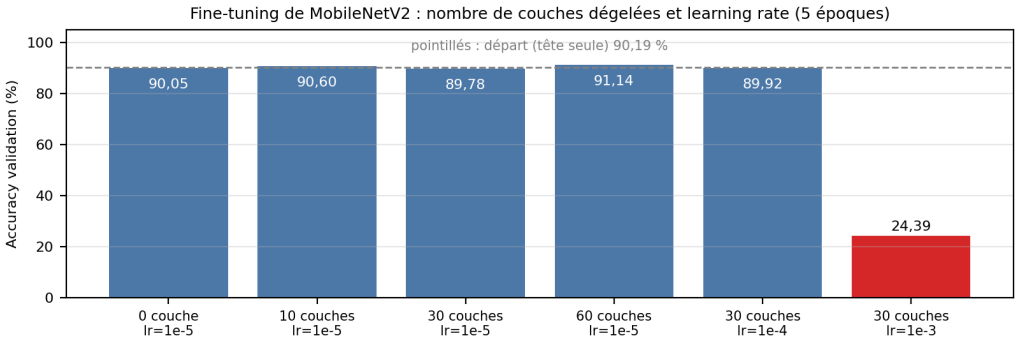


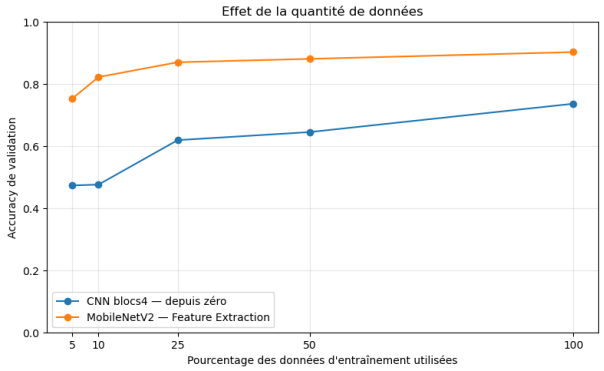
Figure 4 — Accuracy de validation après 5 époques de fine-tuning. Les 5 configurations à lr ≤ 10^{-4} sont dans l'intervalle de bruit (± 2,2 pts).

Couches dégelées	lr	Params entraînables	Val. finale	Train finale	Val. après époque 1
0 (témoin)	1e-5	6 405	90,05 %	96,80 %	89,92 %
10	1e-5	731 845	90,60 %	97,38 %	89,78 %
30	1e-5	1 517 125	89,78 %	98,98 %	89,78 %
60	1e-5	1 938 181	91,14 %	99,25 %	89,78 %
30	1e-4	1 517 125	89,92 %	99,46 %	90,60 %
30	1e-3	1 517 125	**24,39 %**	20,78 %	**24,39 %**

Lecture. (1) De 0 à 60 couches dégelées, l'accuracy reste entre 89,8 % et 91,1 % : aucune différence significative, alors que l'écart entraînement/validation grandit (sur-apprentissage). Sur ~3 000 images d'un domaine proche d'ImageNet, **dégeler peu, voire pas du tout, suffit** ; la règle générale est de dégeler de la fin vers le début, d'autant plus profondément que les données sont nombreuses et le domaine éloigné. (2) Avec lr = 10^{-3}, la validation **s'effondre à 24,4 % dès la 1re époque** (valeur identique à l'époque 1 et à l'époque 5, égale à la part de la classe pissenlit, 179/734, ce qui est cohérent avec une prédiction constante) : les poids pré-entraînés sont détruits (**oubli catastrophique**). Un lr de 10^{-5} à 10^{-4} préserve les filtres génériques ; on entraîne d'abord la tête pour éviter que ses gradients aléatoires n'abîment le tronc.

1.5 Effet de la quantité de données

CNN de zéro et MobileNetV2 (extraction) sont réentraînés avec 5, 10, 25, 50 et 100 % de l'entraînement (sous-ensembles stratifiés ; validation fixe ; 10 époques ; une exécution par point).

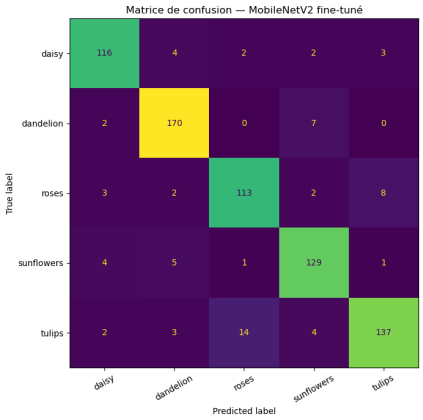


%	Images	CNN zéro	MNetV2	Δ (pts)
5 %	146	47,4 %	75,3 %	+27,9
10 %	293	47,7 %	82,3 %	+34,6
25 %	734	62,0 %	87,1 %	+25,1
50 %	1 468	64,6 %	88,2 %	+23,6
100 %	2 936	73,7 %	90,3 %	+16,6

Figure 5 — Accuracy de validation selon la taille du jeu d'entraînement (Δ = avantage de MobileNetV2 sur le CNN de zéro, en points).

- **Avec 5 % des données (146 images), le transfert (75,3 %) dépasse le CNN de zéro entraîné sur 100 % (73,7 %)** : il est environ 20 fois plus économe en données.
- La courbe du transfert est presque plate (+3,3 pts de 25 à 100 %), celle du CNN monte encore (+11,7 pts) : l'écart se réduit (34,6 → 16,6 pts) mais ne s'annule pas dans la plage testée.
- **Précautions :** une seule exécution par point (la même configuration a donné 72,3 % puis 73,7 % : variabilité ≈ 1 à 1,5 pt) ; à 5 %, le CNN ne fait que ≈ 50 pas de gradient en 10 époques contre 920 à 100 %, donc une partie de sa faiblesse vient du budget d'optimisation et pas seulement du manque d'images.

1.6 Effet de l'oubli de preprocess_input à l'inférence



Prétraitement	Accuracy	Loss
Correct : pixels dans [−1, 1]	90,6 %	0,315
Oublié : pixels dans [0, 1]	86,1 %	0,480
Écart	**−4,5 pts**	**≈ +53 %**

Classe	Précision	Rappel	F1
marguerite	0,91	0,91	0,91
pissenlit	0,92	0,95	0,94
rose	0,87	0,88	0,88
tournesol	0,90	0,92	0,91
tulipe	0,92	0,86	0,89

Figure 6 — Matrice de confusion du modèle fine-tuné (gauche) ; impact du prétraitement et rapport de classification (droite).

L'oubli du prétraitement coûte **4,5 pts** (au-delà de l'incertitude de ± 2,2 pts : l'effet est réel) et augmente la loss d'environ 53 % : le modèle reste souvent juste mais **beaucoup moins confiant et moins bien calibré**. L'erreur est **silencieuse** (aucune exception, bonnes dimensions). Remède : intégrer le prétraitement dans le modèle. L'erreur dominante est tulipe → rose (14 images) puis rose → tulipe (8) : deux fleurs souvent photographiées en bouquet, de couleurs proches. Les F1 des cinq classes sont comprises entre 0,88 et 0,94 : le modèle est homogène.

2. Exercice 2 — Choix d'un backbone pour un déploiement embarqué

Pour VGG16, ResNet50, MobileNetV2 et EfficientNetB0 (entrée 224×224, include_top=False, pooling='avg'), chaque tronc est gelé et utilisé avec **son propre** preprocess_input. Les vecteurs de caractéristiques sont extraits **une seule fois** (images et labels dans le même ordre, vérifié par assert), puis une **régression logistique** scikit-learn est entraînée sur ces vecteurs en quelques secondes. Le modèle Keras sauvegardé reprend les coefficients de la régression (sinon sa tête serait aléatoire).

```
def build_backbone(ctor):   # tronc gelé ; caractéristiques = moyenne globale des cartes finales
    m = ctor(weights="imagenet", include_top=False, pooling="avg", input_shape=(224, 224, 3))
    m.trainable = False;  return m

F_train, y_tr, lat_tr = extract_features(backbone, preprocess, e2_train_ds)   # une seule passe, sans shuffle
F_val,   y_va, lat_va = extract_features(backbone, preprocess, e2_val_ds)
assert np.array_equal(y_tr, e2_y_train) and np.array_equal(y_va, e2_y_val)    # contrôle d'alignement
clf = LogisticRegression(max_iter=3000, random_state=SEED).fit(F_train, y_tr)
acc = accuracy_score(y_va, clf.predict(F_val))
head.set_weights([clf.coef_.T, clf.intercept_])    # modèle déployable = prétraitement + tronc + Dense softmax
# latence « propre » : tf.function, tenseur en mémoire, médiane après échauffement (batch = 1 et 32)
```

2.1 Tableau comparatif

Backbone	Params tronc	Dim.	Accuracy val.	Latence extr. (ms/img)[1]	Latence propre b=1 (ms)	Latence propre b=32 (ms/img)	Taille (Mo)
EfficientNetB0	4 049 571	1 280	**93,87 %**	49,3	28,0	13,6	16,3
ResNet50	23 587 712	2 048	91,83 %	68,0	89,1	46,8	90,7
MobileNetV2	2 257 984	1 280	89,51 %	15,7	**19,5**	**10,1**	**9,2**
VGG16	14 714 688	512	86,65 %	164,8	90,9	125,6	56,2

[1] *Latence mesurée en parcourant le tf.data.Dataset : elle inclut la lecture et le décodage JPEG, donc peu fiable. Les colonnes « propre » mesurent le modèle seul (médiane, CPU de PC, pas de GPU ni de smartphone).*

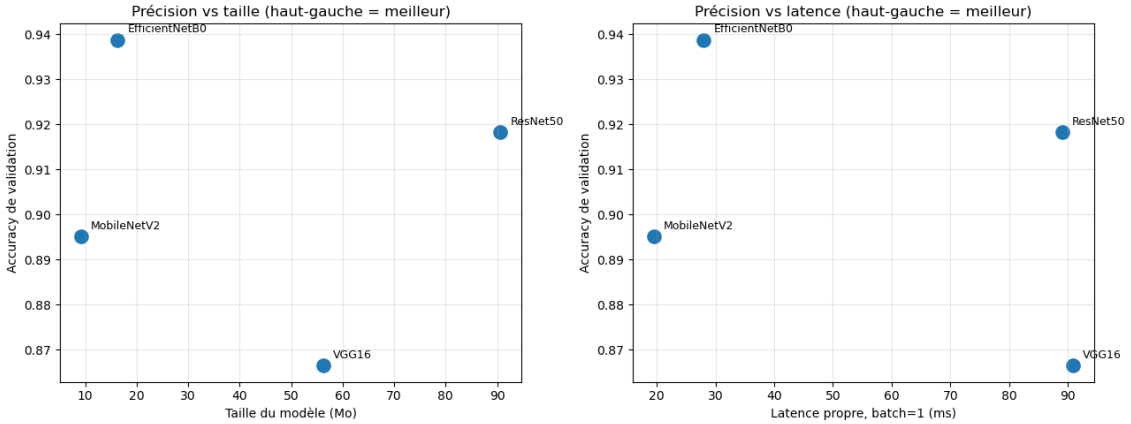


Figure 7 — Compromis précision / taille (gauche) et précision / latence propre à batch = 1 (droite). Haut-gauche = meilleur.

- **Plus de paramètres ne veut pas dire meilleure précision.** EfficientNetB0 (4,05 M) est le meilleur ; ResNet50 a 5,8 fois plus de paramètres pour 2,0 pts de moins ; VGG16 (14,7 M) est dernier ; MobileNetV2 (2,26 M) dépasse VGG16 de

2,9 pts. L'efficacité de l'architecture (convolutions séparables, blocs résiduels, mise à l'échelle conjointe) compte plus que la taille brute.

- La **taille sur disque** suit les paramètres (≈ 4 octets chacun, float32) ; la tête logistique pèse 0,02 à 0,08 Mo : tout le coût de déploiement est dans le tronc. La dimension des caractéristiques ne prédit pas la qualité (VGG16 : 512 dimensions, le moins précis ; ResNet50 : 2 048, mais derrière EfficientNetB0 : 1 280).
- **Front de Pareto :** EfficientNetB0 et MobileNetV2. ResNet50 et VGG16 sont **dominés** (un autre modèle est à la fois plus précis, plus petit et plus rapide).
- **Significativité :** EfficientNetB0 / ResNet50 (+2,0 pts) est à la limite du bruit (± 2,2 pts) ; EfficientNetB0 / MobileNetV2 (+4,4) et / VGG16 (+7,2) sont nets. MobileNetV2 + régression logistique en 224×224 (89,5 %) et MobileNetV2 + Dense en 160×160 de l'exercice 1 (90,2 %) sont équivalents : le choix du tronc compte plus que celui de la tête ou de la résolution.

2.2 Séparabilité des classes : t-SNE et matrices de confusion

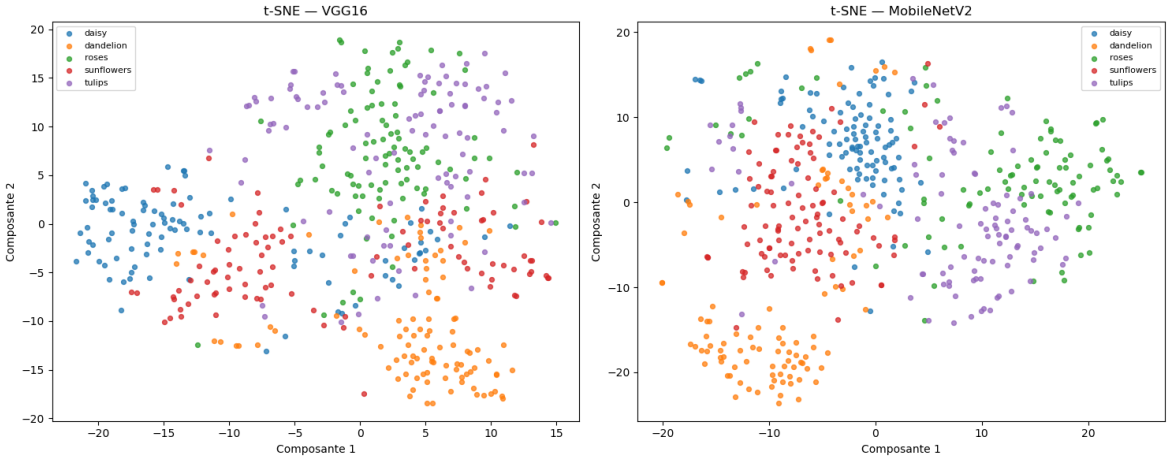
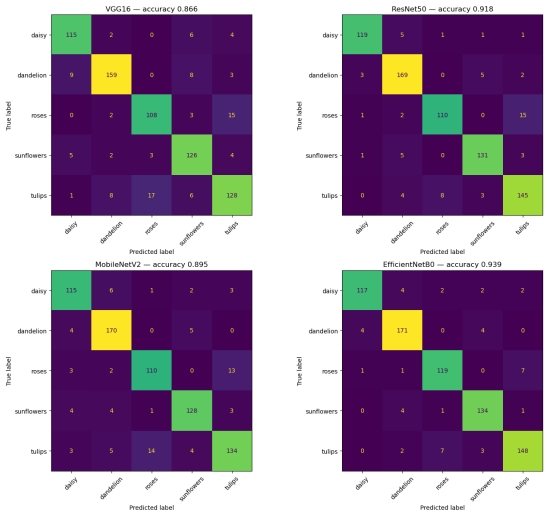


Figure 8 — t-SNE de 500 images de validation (100 par classe) : VGG16 (gauche) et MobileNetV2 (droite).



Backbone	rose → tulipe	tulipe → rose
VGG16	15	17
ResNet50	15	8
MobileNetV2	13	14
EfficientNetB0	**7**	**7**

Pissenlit forme l'amas le plus compact et le plus isolé. **Rose et tulipe se chevauchent** fortement, surtout avec VGG16 ; MobileNetV2 les mélange moins. Marguerite et tournesol sont séparées mais plus diffuses (images plus variées). EfficientNetB0 divise par deux la confusion rose ↔ tulipe, ce qui explique l'essentiel de son avance. **Limite :** t-SNE préserve les voisinages locaux mais déforme les distances globales ; un recouvrement en 2D ne prouve pas la non-séparabilité en 1 280 dimensions. Il se lit avec les matrices de confusion, jamais seul.

Figure 9 — Matrices de confusion des quatre backbones (gauche) et erreurs rose ↔ tulipe (droite).

2.3 Prédictions Top-1 / Top-5 avec la tête ImageNet complète



Image (classe réelle)	VGG16	ResNet50	MobileNetV2	EfficientNetB0
Marguerite	daisy (0,96)	daisy (0,98)	daisy (0,69)	daisy (0,94)
Pissenlit	buckeye (0,33)	sea_urchin (0,37)	porcupine (0,35)	daisy (0,21)
Rose	altar (0,95)	altar (0,78)	altar (0,75)	plate_rack (0,28)
Tournesol	daisy (0,40)	daisy (0,34)	daisy (0,88)	daisy (0,74)
Tulipe	vase (0,92)	vase (0,99)	vase (0,93)	vase (0,94)

Tableau — Top-1 (score softmax) pour une image de validation par classe.

La tête ImageNet classe parmi **1 000 catégories** dont une seule correspond à nos fleurs (*daisy*). Les prédictions sont « justes » uniquement pour la marguerite, car la classe existe. Pour les autres, le modèle retombe sur ce qui y ressemble : objets ronds et épineux pour le pissenlit (*buckeye*, *sea_urchin*, *porcupine*), **contexte** d'intérieur pour la rose (*altar*, *plate_rack*), fleur la plus proche connue pour le tournesol (*daisy*), **objet dominant** pour la tulipe (*vase*, 0,92 à 0,99). Le Top-5 ne rattrape rien quand le concept est absent de l'espace d'étiquettes. Les scores sont trompeurs (*vase* à 0,99, *altar* à 0,95) : un softmax reste très confiant hors de son domaine. Pourtant, les caractéristiques internes de ces mêmes réseaux séparent très bien les fleurs (87 à 94 % avec une simple régression logistique) : c'est tout l'intérêt du **transfert inductif** (garder le tronc, remplacer la tête). Une seule image par classe est testée : cela illustre le mécanisme sans constituer une mesure statistique.

2.4 Recommandations de déploiement

Scénario	Backbone	Justification	Précautions
(a) Serveur cloud avec GPU	**EfficientNetB0**	Meilleure accuracy (93,9 %), plus petit et plus rapide que ResNet50 et VGG16 ; sur GPU les écarts de latence se compressent, la précision devient le critère dominant.	ResNet50 en alternative mature (2 pts de moins, 5,6× plus lourd). Essayer EfficientNet B3–B4 et le fine-tuning (non testés).
(b) Smartphone d'entrée de gamme, hors ligne	**MobileNetV2**	Le plus compact (9,2 Mo, 2,26 M de paramètres) et le plus rapide (19,5 ms/img) ; conçu pour le mobile, adapté à la quantification int8.	−4,4 pts vs EfficientNetB0 en extraction pure, atténuables (fine-tuning, entrée 160×160 : ≈ 90 %). Conversion TFLite + int8 ; valider sur l'appareil réel.
À écarter	VGG16	Dominé : le moins précis, 6× plus lourd que MobileNetV2, très lent.	—

Limites : latences mesurées sur CPU de PC, pas sur ARM ni sur GPU : la décision finale doit s'appuyer sur un benchmark sur l'appareil cible. L'écart EfficientNetB0 / ResNet50 est à la limite du bruit (une validation croisée le confirmerait). Aucun fine-tuning des backbones n'a été fait ici.

3. Exercice 3 — Apprentissage multitâche pour un catalogue e-commerce

3.1 Données, étiquettes de la tâche B et architecture

Fashion-MNIST (28×28, 54 000 images d'entraînement, 6 000 de validation, 10 000 de test officiel réservé à l'évaluation finale). Tâche A : catégorie (10 classes). Tâche B : famille (4 classes), construite avec famille = [0, 1, 0, 1, 0, 2, 0, 2, 3, 2] ; une table de contingence (vérifiée par assert) confirme que chaque catégorie tombe dans une seule famille. B est donc une **fonction déterministe de A** et est déséquilibrée (Hauts 40 %, Chaussures 30 %, Bas et robes 20 %, Accessoires 10 % : classe majoritaire = 40 %). B n'apporte aucune information nouvelle sur l'image, seulement une supervision plus grossière : on n'attend pas de gain spectaculaire de précision, mais un gain de coût.

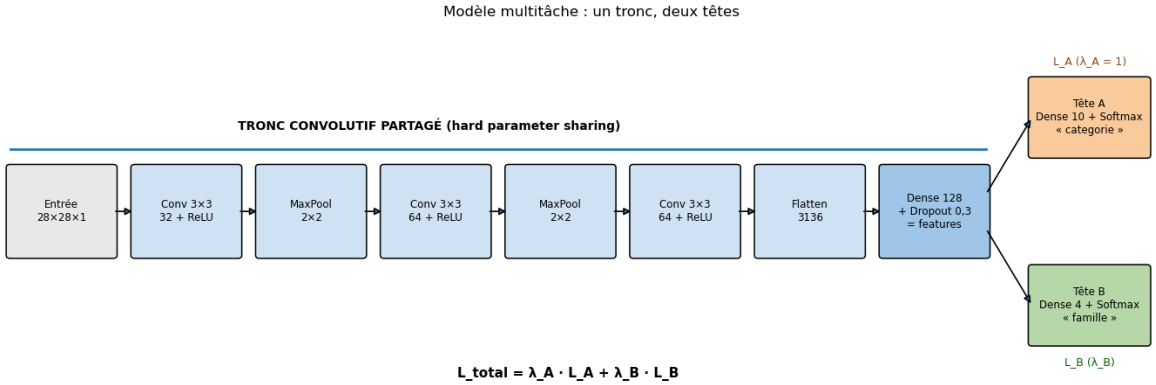


Figure 10 — Schéma de l'architecture multitâche : tronc convolutif partagé (hard parameter sharing), une tête softmax par tâche, $L_{total} = \lambda_A \cdot L_A + \lambda_B \cdot L_B$.

```
famille = np.array([0, 1, 0, 1, 0, 2, 0, 2, 3, 2]);  yb_tr = famille[ya_tr]          # étiquettes de la tâche B
def build_trunk(inputs):           # tronc partagé (identique pour tous les modèles) : 28x28 -> 14 -> 7 -> 3136 -> 128
    x = layers.Conv2D(32, 3, padding="same", activation="relu")(inputs); x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x);      x = layers.MaxPooling2D(2)(x)
    x = layers.Conv2D(64, 3, padding="same", activation="relu")(x);      x = layers.Flatten()(x)
    x = layers.Dense(128, activation="relu")(x);  return layers.Dropout(0.3)(x)       # = features

features = build_trunk(inputs)
out_a = layers.Dense(10, activation="softmax", name="categorie")(features)
out_b = layers.Dense(4,  activation="softmax", name="famille")(features)
model = keras.Model(inputs, {"categorie": out_a, "famille": out_b})
model.compile(optimizer=keras.optimizers.Adam(1e-3),
              loss={"categorie": "sparse_categorical_crossentropy", "famille": "sparse_categorical_crossentropy"},
              loss_weights={"categorie": 1.0, "famille": lambda_b}, metrics={"categorie": ["accuracy"], "famille": ["accuracy"]})
# Variante « tête B dérivée » : p_B = p_A · M  (Dense 10->4 figée, M[c, famille[c]] = 1)
# Décodage conjoint : c* = argmax_c  log pA(c) + log pB(famille(c))   |   hiérarchique : argmax pB, puis argmax pA dans la famille
```

Paramètres du multitâche : **459 086** au total = tronc partagé 457 280 + tête catégorie 1 290 + tête famille 516. Entraînement : Adam 1e-3 ; 8 époques, batch 128 (données complètes) ; 3 000 images : 30 époques, batch 64. Trois graines par configuration.

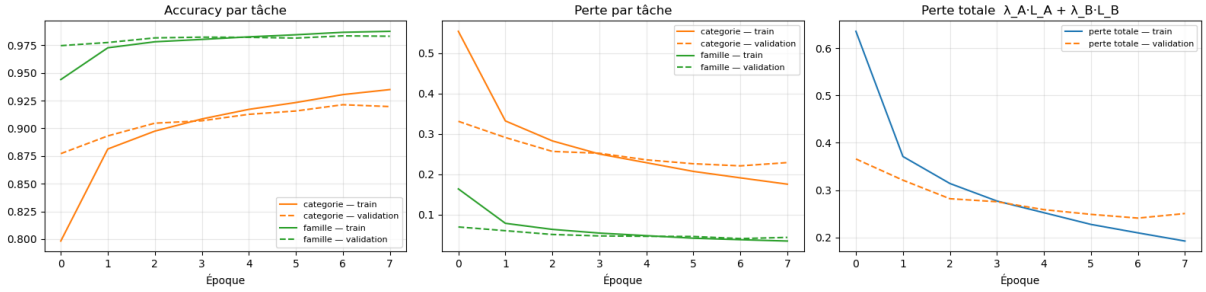


Figure 11 — Courbes d'apprentissage du multitâche (λ_B = 0,5, graine 42) : accuracy et perte par tâche, perte totale.

3.2 Mono-tâche vs multitâche (données complètes, λ_A = 1, λ_B = 0,5)

Configuration	Params	Acc. A (%)	Acc. B (%)	Latence b=256 (ms/img)	Latence b=1 (ms/img)
Mono A (catégorie)	458 570	91,58 ± 0,25	—	0,0632	1,065
Mono B (famille)	457 796	—	98,09 ± 0,06	0,1103	0,945
Mono A + Mono B (2 modèles)	**916 366**	91,58 ± 0,25	98,09 ± 0,06	**0,1735**	**2,010**
Multitâche (λ_B = 0,5)	**459 086**	91,48 ± 0,32	98,10 ± 0,06	**0,1077**	**1,013**
Multitâche, tête B dérivée	458 610	91,41 ± 0,18	98,12 ± 0,04	0,1081	0,978

Précisions : moyenne ± écart-type sur 3 graines, jeu de test (10 000 images). Les deux mono-tâches doivent être exécutés ensemble pour obtenir les deux étiquettes : leur coût s'additionne.

- **Coût :** −50 % de paramètres (459 086 vs 916 366) ; latence −49,6 % à batch = 1 (1,013 vs 2,010 ms : une passe au lieu de deux) et −38,0 % à batch = 256. Les mesures de latence sont bruitées (Mono B plus lent que Mono A à batch = 256

malgré un tronc identique) : seul l'ordre de grandeur est fiable.

- **Précision :** Δ(multi − mono) = −0,10 ± 0,45 pt sur A et +0,01 ± 0,02 pt sur B : **différences dans le bruit inter-graines**. Un résultat « multi ≈ mono » est le résultat normal ici : avec 54 000 images, rien ne manque aux mono-tâches et B se déduit de A. On obtient la même qualité pour moitié moins de coût.

3.3 Influence de $\lambda_B \in \{0{,}1 ; 0{,}5 ; 1 ; 2 ; 5\}$ ($\lambda_A = 1$)

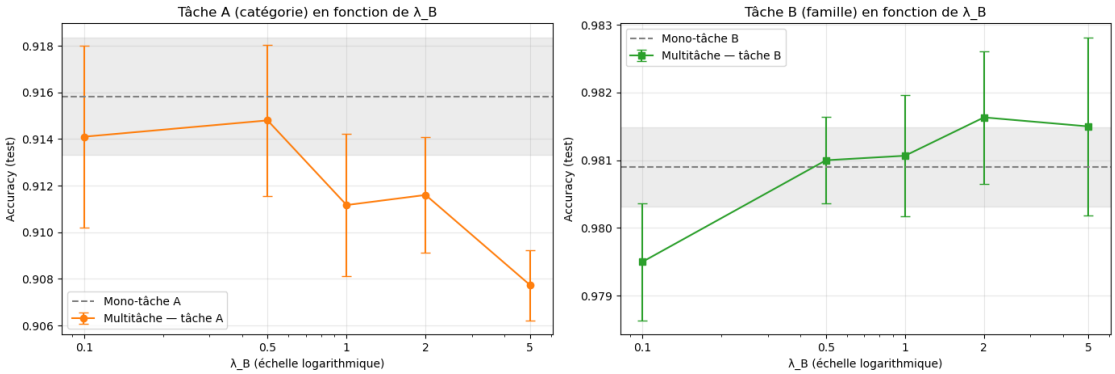


Figure 12 — Accuracy de test des tâches A et B selon λ_B (échelle log, barres = écart-type sur 3 graines ; pointillés et bande grise = mono-tâche).

λ_B	0,1	0,5	1	2	5	Mono-tâche
Acc. A (%)	91,41 ± 0,39	91,48 ± 0,32	91,12 ± 0,31	91,16 ± 0,25	**90,77 ± 0,15**	91,58 ± 0,25
Acc. B (%)	97,95 ± 0,09	98,10 ± 0,06	98,11 ± 0,09	98,16 ± 0,10	98,15 ± 0,13	98,09 ± 0,06

La tâche A varie de 0,71 pt sur la plage de λ_B (bruit inter-graines ≈ 0,28 pt). À partir de $\lambda_B = 1$, A passe sous la référence mono-tâche ; à $\lambda_B = 5$ elle perd 0,8 pt (≈ 3 écarts-types) : c'est un **léger transfert négatif**. Mécanisme : une forte pondération de la perte B, facile (4 classes), fait dominer son gradient dans le tronc partagé, qui s'oriente vers la discrimination grossière des familles au détriment de la finesse des catégories. B ne gagne rien au-delà de $\lambda_B \approx 0{,}5$ (variation de 0,2 pt, dans le bruit) : rendements décroissants. Le meilleur compromis est $\boldsymbol{\lambda_B \approx 0{,}5}$. Avec seulement 3 graines, ces écarts sub-pour-cent restent à confirmer.

3.4 Régime « peu de données » : 3 000 images d'entraînement (300 par catégorie)

Configuration	Acc. A (%)	Acc. B (%)
Mono	85,77 ± 0,43	96,79 ± 0,24
Multi ($\lambda_B = 0{,}5$)	85,99 ± 0,05	96,64 ± 0,22
Multi ($\lambda_B = 1$)	86,19 ± 0,45	96,65 ± 0,09
Multi, tête B dérivée	85,71 ± 0,58	96,77 ± 0,19

Régime	λ_B	Δ A (pt)	Δ B (pt)
Complet	0,5	−0,10 ± 0,45	+0,01 ± 0,02
Complet	1	−0,47 ± 0,23	+0,02 ± 0,08
3 000 img.	0,5	+0,22 ± 0,41	−0,15 ± 0,05
3 000 img.	1	+0,42 ± 0,63	−0,14 ± 0,17

Tableaux — Accuracy de test avec 3 000 images (gauche) ; écarts appariés multi − mono selon le régime (droite).

Avec 3 000 images, l'accuracy de A chute à ≈ 86 % (contre 91,6 %) et devient plus variable. Le multitâche gagne +0,2 à +0,4 pt sur A, mais **moins que l'écart-type** : le gain n'est pas démontré, et la tâche B perd légèrement (−0,15 pt, au seuil du bruit). **Le partage de représentation n'aide donc pas nettement plus avec peu de données dans ce montage.** Explication probable : B étant un regroupement de A, la supervision supplémentaire est faible ; l'effet de régularisation attendu existe peut-être mais reste sous le bruit de l'expérience.

3.5 Prédictions incohérentes et solutions

Une prédiction est **incohérente** quand argmax(B) ≠ famille[argmax(A)] (ex. A prédit « Pantalon » et B « Chaussures »).

Modèle (décisions brutes)	Données complètes	3 000 images
2 mono-tâches	1,43 ± 0,08 %	2,18 ± 0,18 %
Multitâche (têtes indépendantes)	0,64 ± 0,02 %	0,95 ± 0,15 %
Multitâche, tête B dérivée	**0,23 ± 0,05 %**	**0,17 ± 0,03 %**

Où surviennent-elles ? Sur les images ambiguës : confiance moyenne de la tête A 0,52 pour les cas incohérents contre 0,94 pour les cohérents (tête B : 0,63 contre 0,99) ; A est fausse dans 61 % de ces cas et B dans 44 %. **77,4 %** des erreurs de A restent dans la même famille (chemise → T-shirt 107, chemise → pull 86, chemise → manteau 79, manteau → pull 77) : elles ne créent aucune incohérence puisque B répond « Hauts ».

Tableau — Taux d'incohérence (gauche) et analyse des cas incohérents, graine 42 (droite).

Stratégie (modèle)	Incohérence	Acc. A (%)	Acc. B (%)	Remarque
Brut — 2 mono-tâches	1,43 %	91,58	98,09	Référence
Dérivation B ← A (2 mono)	0 %	91,58	98,16	Gratuit ; B dépend de A
Décodage conjoint (2 mono)	**0 %**	**91,71**	**98,32**	Meilleur ; sans réentraînement
Décodage hiérarchique (2 mono)	0 %	91,51	98,09	Une erreur de famille se propage à A
Brut — multitâche	0,64 %	91,48	98,10	—
Décodage conjoint (multitâche)	0 %	91,47	98,14	Sans perte de précision
Tête B dérivée + hiérarchique	0 %	91,34	98,12	Cohérent par construction

Solution proposée. Les têtes sont des classifieurs indépendants sur un même tronc : rien ne les oblige à s'accorder, d'où un taux faible mais non nul, plus élevé pour deux mono-tâches entraînés séparément (1,43 %) que pour le multitâche (0,64 %). Le plus simple et le plus robuste est un **décodage conjoint** (choisir la catégorie qui maximise $\log p_A(c) + \log p_B(\text{famille}(c))$, puis déduire la famille) : **0 % d'incohérence, aucune perte de précision** et même un petit gain sur B, sans réentraîner. Alternatives : une **tête B dérivée** ($p_B = p_A \cdot M$, -516 paramètres, incohérence brute ramenée à 0,23 %, nulle avec le décodage hiérarchique), un terme de cohérence (divergence KL entre p_B et $p_A \cdot M$) dans la perte, ou le rejet des cas incohérents (« photo incertaine »). **Limite :** cette correspondance est parfaite car chaque catégorie appartient à une seule famille ; des familles qui se recoupent exigeraient une formulation multi-étiquette.

4. Analyse comparative et recommandations

4.1 Tableau récapitulatif : de zéro, extraction, fine-tuning

Approche	Époques	Params entraînables	Acc. validation	Acc. avec 5 % des données	Observation
CNN blocs4 depuis zéro	10	913 221	72,3 %	47,4 %	Sous-appris ; très dépendant des données
MobileNetV2 — extraction (gelé)	10	6 405	**90,2 %**	75,3 %	Meilleur rapport coût / précision ; converge dès l'époque 1
MobileNetV2 — fine-tuning (30 couches)	10 + 10	1 517 125	**90,6 %**	non mesuré	+0,4 pt non significatif ; train 99,6 % (sur-apprentissage)

4.2 Réponses aux six questions d'analyse transversale

Q1 — Quantité de données. Dans nos expériences l'avantage du transfert **ne s'estompe pas** jusqu'à 2 936 images : il passe de +34,6 pts (10 %) à +16,6 pts (100 %), soit une réduction de moitié. Le transfert à 146 images (75,3 %) bat le CNN de zéro à 2 936 images (73,7 %). Les courbes ne se rejoignent qu'avec beaucoup plus de données ou d'époques (ordre : dizaines de milliers d'images), point non atteint ici ; toute extrapolation serait spéculative.

Q2 — Gel et dégel. Dégeler 0, 10, 30 ou 60 couches donne 89,8 à 91,1 % : pas de différence significative, mais un sur-apprentissage croissant ; sur ce domaine proche d'ImageNet, dégeler peu suffit. Le **learning rate faible** est indispensable : 10^{-5} et 10^{-4} préservent les poids pré-entraînés, 10^{-3} fait chuter la validation à 24,4 % dès la 1re époque (oubli catastrophique). On entraîne d'abord la tête, puis on dégèle avec un petit pas, BatchNorm gelées.

Q3 — Compromis. Une meilleure précision n'implique pas plus de paramètres : EfficientNetB0 (4,05 M, 93,9 %) bat ResNet50 (23,6 M, 91,8 %) et VGG16 (14,7 M, 86,7 %). La taille suit les paramètres (9 à 91 Mo) ; la latence, elle, n'est pas prédite par le nombre de paramètres : à batch = 1, ResNet50 (23,6 M) et VGG16 (14,7 M) ont des latences voisines (89 et 91 ms) ; à batch = 32, VGG16 est 9,3 fois plus lent qu'EfficientNetB0 (125,6 vs 13,6 ms/img) pour 3,6 fois plus de paramètres. Les convolutions séparables de MobileNetV2 et EfficientNetB0 sont très économes en calcul. Même constat sur ImageNet (énoncé) : EfficientNetB0 (5,3 M, 77,1 %) bat VGG16 (138 M, 71,3 %).

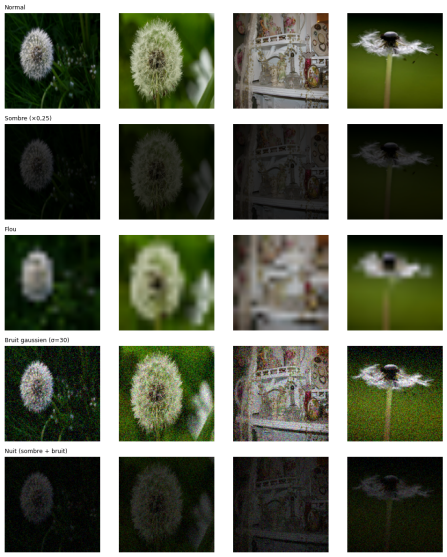
Q4 — Prétraitement. Un prétraitement incohérent coûte 4,5 pts d'accuracy et ≈ 53 % de loss en plus, **sans aucune erreur visible** ; chaque backbone a le sien (VGG16/ResNet50 : moyenne ImageNet en BGR ; MobileNetV2 : [−1, 1] ; EfficientNet de Keras : normalisation intégrée). Remède : l'intégrer au modèle, versionner l'ensemble et tester le modèle exporté sur des images de référence.

Q5 — Multitâche. Il a aidé sur le **coût** (−50 % de paramètres, −38 à −50 % de latence) à précision équivalente, et sur la **cohérence** des sorties (0,64 % contre 1,43 % d'incohérence). Il n'a pas amélioré la précision de façon démontrée, y compris avec 3 000 images. Un **transfert négatif léger** apparaît pour $\lambda_B \geq 1$ (A : −0,8 pt à $\lambda_B = 5$). Conditions favorables : tâches liées, pondérations équilibrées, tronc assez large. Dans ce montage B se déduit de A et n'apporte aucune information nouvelle.

Q6 — Décalage de distribution. Voir l'expérience ci-dessous : le modèle de fleurs **n'est pas fiable** sur photos sombres, floues ou bruitées.

4.3 Expérience pour la question 6 : robustesse du modèle fine-tuné

Les mêmes 734 images de validation sont dégradées ; les étiquettes ne changent pas (P(y | x) identique), seule la distribution P(x) se déplace : c'est la définition du **covariate shift** (équation 1 de l'énoncé).



Condition	Accuracy	Loss	Chute (pts)
Normal (référence)	90,6 %	0,315	—
Sombre (×0,25)	79,7 %	0,733	10,9
Flou (16×16 → 160×160)	48,9 %	1,812	**41,7**
Bruit gaussien (σ = 30)	70,4 %	1,093	20,2
Nuit (sombre + bruit σ = 12)	43,1 %	2,215	**47,5**

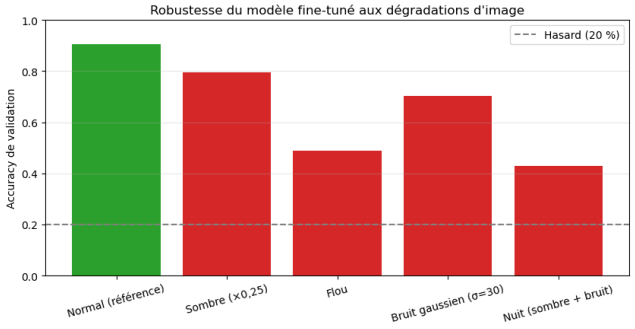


Figure 13 — Exemples de dégradations (gauche ; lignes : normal, sombre, flou, bruit, nuit) et accuracy par condition (droite). Hasard = 20 %.

L'assombrissement seul est toléré (−10,9 pts : la structure de l'image subsiste). Le **flou** et le **bruit** détruisent les hautes fréquences (textures, contours des pétales) dont dépendent les premières couches et la distinction rose/tulipe ; le cas « nuit

» combine les deux et tombe à 43 %. L'hypothèse du covariate shift « bénin » (même support) est fragile : des images très sombres et bruitées peuvent sortir du support de l'entraînement, et le modèle extrapole alors sans garantie. S'y ajoute un décalage de domaine (photos Flickr bien exposées → smartphone en basse lumière). **Remèdes :** augmentation photométrique (luminosité, contraste, flou, bruit, compression JPEG) ; collecte d'images du domaine cible et fine-tuning, même avec peu d'exemples ; adaptation de domaine (BatchNorm adaptatives, apprentissage adversarial) ; prétraitement à l'inférence (débruitage, égalisation d'histogramme) ; surveillance en production (statistiques de luminosité, entropie des prédictions) et rejet des entrées incertaines (« reprenez la photo »), sachant que le softmax est sur-confiant hors domaine (§ 2.3).

4.4 Recommandation de backbone et apports/limites du multitâche

Recommandation. (a) Serveur avec GPU : **EfficientNetB0**. (b) Smartphone d'entrée de gamme, hors ligne : **MobileNetV2** (conversion TFLite, quantification int8, entrée 160×160), à valider sur l'appareil réel. VGG16 est à écarter. Les deux modèles ont un vecteur de 1 280 dimensions : une base de code commune est possible, les têtes étant interchangeables.

Apports du multitâche	Limites du multitâche
• Un seul tronc : coût d'inférence et de stockage divisés par ≈ 2 (décisif sur mobile) • Effet de régularisation possible, surtout avec peu de données (non démontré ici) • Cohérence entre sorties plus facile à garantir (tête dérivée, décodage conjoint) • Un seul modèle à entraîner, versionner et surveiller	• Gain de précision non garanti : ici B se déduit de A, donc « multi ≈ mono » • Choix de λ délicat (échelle des pertes) ; transfert négatif possible ($\lambda_B = 5$) • Couplage : modifier une tâche impose de réentraîner le tronc et peut dégrader l'autre • Incohérences possibles entre têtes indépendantes ; évaluation plus coûteuse (plusieurs graines)

4.5 Limitations observées et pistes d'amélioration

Limitation observée	Piste d'amélioration
Validation de 734 images (± 2,2 pts), une seule exécution pour la plupart des points des exercices 1 et 2	Validation croisée ou plusieurs graines ; intervalles de confiance ; test de McNemar
CNN de zéro sous-appris (10 époques) ; budget d'optimisation inégal selon la taille du jeu	Plus d'époques, scheduler, early stopping ; comparaison à budget d'optimisation égal
Fine-tuning : sur-apprentissage, gain non significatif	Augmentation de données sur MobileNetV2, régularisation plus forte, dégel progressif, label smoothing
Confusion systématique rose ↔ tulipe	Plus de données sur ces classes, résolution plus élevée, modèle plus grand (EfficientNet B3+)
Latences mesurées sur CPU de PC, bruitées	Benchmark sur l'appareil cible, TFLite + quantification
Prédictions ImageNet hors sujet et sur-confiantes ; fragilité au flou, au bruit et à la nuit	Remplacer la tête, calibrer (temperature scaling), seuil de rejet, augmentation photométrique, adaptation de domaine
Multitâche : B déduit de A ; seulement 3 graines	Tâches complémentaires (couleur, motif, genre) ; perte de cohérence KL ; pondération automatique (incertitude, GradNorm) ; plus de graines